$\{1, \ldots, m\}$ a Q-factor value $Q_k(b_k, u_{k+1})$ by simulating the base policy for multiple time steps starting from all possible posteriors b_{k+1} that can be generated from (b_k, u_{k+1}), and by accumulating the corresponding cost [including a terminal cost such as $G(b_N)$]; see Fig. 2.10.5. It then selects the next point $\tilde{u}_{k+1}$ for observation by using the Q-factor minimization of Eq. (2.98).

Note that the equation

$$b_{k+1} = B_k(b_k, u_{k+1}, z_{u_{k+1}}), \qquad k = 0, \ldots, N-1,$$

which governs the evolution of the posterior distribution (or belief state), is stochastic because $z_{u_{k+1}}$ involves the stochastic noise $w_{u_{k+1}}$. Thus some Monte Carlo simulation is unavoidable in the calculation of the Q-factors $Q_k(b_k, u_{k+1})$. On the other hand, one may greatly reduce the Monte Carlo computational burden by employing a certainty equivalence approximation, which at stage k, treats only the noise $w_{u_{k+1}}$ as stochastic, and replaces the noise variables $w_{u_{k+2}}, w_{u_{k+3}}, \ldots$, after the first stage of the calculation, by deterministic quantities such as their means $\hat{w}_{u_{k+2}}, \hat{w}_{u_{k+3}}, \ldots$.

The simulation of the Q-factor values may also involve other approximations, some of which have been suggested in various proposals for rollout-based BO. For example, if the number of possible observations m is very large, we may compute and compare the Q-factors of only a subset. In particular, at a given time k, we may rank the observations by using an acquisition function, select a subset U_{k+1} of most promising observations, compute their Q-factors $Q_k(b_k, u_{k+1})$, $u_{k+1} \in U_{k+1}$, and select the observation whose Q-factor is minimal; this idea has been used in the case of the Wordle puzzle in the papers by Bhambri, Bhattacharjee, and Bertsekas [BBB22], [BBB23], which will be discussed in the next section.

Multiagent Rollout for Bayesian Optimization

In some BO applications there arises the possibility of simultaneously performing multiple observations before receiving feedback about the corresponding observation outcomes. This occurs, among others, in two important contexts:

(a) In parallel computation settings, where multiple processors are used to perform simultaneously expensive evaluations of the function f at multiple points u. These evaluations may involve some form of truncated simulation, so they yield evaluations of the form $z_u = \theta_u + w_u$, where w_u is the simulation noise.

(b) In distributed sensor systems, where a number of sensors provide in parallel relevant information about the random vector θ that we want to estimate; see e.g., the recent paper by Li, Krakow, and Gopalswamy [LKG21], which describes related multisensor estimation problems, based on the multiagent rollout methodology of Section 2.9.

Of course in such cases we may treat the entire set of simultaneous observations as a single observation within an enlarged Cartesian product space of observations, but there is a fundamental difficulty: the size of the observation space (and hence the number of Q-factors to be calculated by rollout at each time step) grows exponentially with the number of simultaneous observations. This in turn greatly increases the computational requirements of the rollout algorithm.

To address this difficulty, we may employ the methodology of multi-agent rollout whereby the policy improvement is done one-agent-at-a-time in a given order, with (possibly partial) knowledge of the choices of the preceding agents in the order. As a result, the amount of computation for each policy improvement grows linearly with the number of agents, as opposed to exponentially for the standard all-agents-at-once method. At the same time the theoretical cost improvement property of the rollout algorithm can be shown to be preserved, while the empirical evidence suggests that great computational savings are achieved with hardly any performance degradation.

Generalization to Sequential Estimation of Random Vectors

Aside from BO, there are several other types of simple sequential estimation problems, which involve "independent sampling," i.e., problems where the choice of an observation type does not affect the quality, cost, or availability of observations of other types. A common class of problems that contains BO as a special case and admits a similar treatment, is to sequentially estimate an m-dimensional random vector $\theta = (\theta_1, \ldots, \theta_m)$ by using N linear observations of θ of the form

$$z_u = a_u'\theta + w_u, \qquad u \in \{1, \ldots, n\},$$

where n is some integer. Here w_u are independent random variables with given probability distributions, the m-dimensional vectors a_u are known, and $a_u'\theta$ denotes the inner product of a_u and θ. Similar to the case of BO, the problem simplifies if the given a priori distribution of θ is Gaussian, and the random variables w_u are independent and Gaussian. Then, the posterior distribution of θ, given any subset of observations, is Gaussian (thanks to the linearity of the observations), and can be calculated in closed form.

Observations are generated sequentially at times $1, \ldots, N$, one at a time and with knowledge of the outcomes of the preceding observations, by choosing an index $u_k \in \{1, \ldots, n\}$ at time k, at a cost $c(u_k)$. Thus u_k are the optimization variables, and affect both the quality of estimation of θ and the observation cost. The objective, roughly speaking, is to select N observations to estimate θ in a way that minimizes an appropriate cost function; for example, one that penalizes some form of estimation

error plus the cost of the observations. We can similarly formulate the corresponding optimization problem in terms of N-stage DP, and develop rollout algorithms for its approximate solution.

2.11 ADAPTIVE CONTROL BY ROLLOUT WITH A POMDP FORMULATION

In this section, we discuss various approaches for the approximate solution of Partially Observed Markovian Decision Problems (POMDP) with a special structure, which is well-suited for adaptive control, as well as other contexts that involve search for a hidden object.† It is well known that POMDP are among the most challenging DP problems, and nearly always require the use of approximations for (suboptimal) solution.

The application and implementation of rollout and approximate PI methods to general finite-state POMDP is described in the author's RL book [Ber19a] (Section 5.7.3). Here we will focus attention on a special class of POMDP where the state consists of two components:

(a) A perfectly observed component x_k that evolves over time according to a discrete-time equation.

(b) An unobserved component θ that stays constant and is estimated through the perfect observations of the component x_k.

We view θ as a parameter in the system equation that governs the evolution of x_k, hence the connection with adaptive control. Thus we have

$$x_{k+1} = f_k(x_k, \theta, u_k, w_k), \tag{2.100}$$

where u_k is the control at time k, selected from a set $U_k(x_k)$, and w_k is a random disturbance with given probability distribution that depends on (x_k, θ, u_k). We will assume that θ can take one of m known values $\theta^1, \ldots, \theta^m$:

$$\theta \in \{\theta^1, \ldots, \theta^m\},$$

see Fig. 2.11.1.

The a priori probability distribution of θ is given and is updated based on the observed values of the state components x_k and the applied controls u_k. In particular, we assume that the information vector

$$I_k = \{x_0, \ldots, x_k, u_0, \ldots, u_{k-1}\}$$

† In Section 1.6.8, we discussed the indirect adaptive control approach, which enforces a separation of the controller into a system identification algorithm and a policy reoptimization algorithm. The POMDP approach of this section (also summarized in Section 1.6.6), does not assume such an a priori separation, and is thus founded on a more principled algorithmic framework.

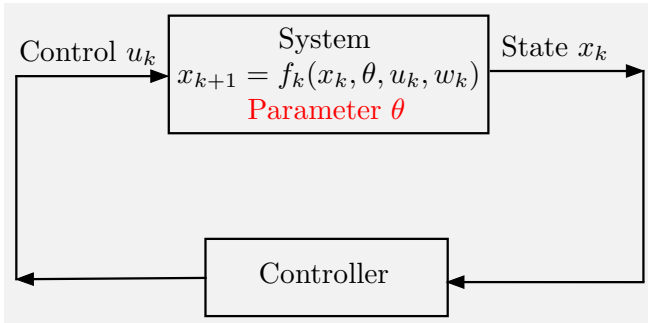

Figure 2.11.1 Illustration of an adaptive control scheme involving perfect state observation of a system with an unknown parameter θ. At each time a decision is made to select a control and (possibly) one of several observation types, each of different cost.

is available at time k, and is used to compute the conditional probabilities

$$b_{k,i} = P\{\theta = \theta^i \mid I_k\}, \qquad i = 1, \ldots, m.$$

These probabilities form a vector

$$b_k = (b_{k,1}, \ldots, b_{k,m}),$$

which together with the perfectly observed state x_k, form the pair (x_k, b_k) that is commonly called the *belief state* of the POMDP at time k.

Note that according to the classical methodology of POMDP (see e.g., [Ber17a], Chapter 4), the belief component b_{k+1} is determined by the belief state (x_k, b_k), the control u_k, and the observation obtained at time $k+1$, i.e., x_{k+1}. Thus b_k can be updated according to an equation of the form

$$b_{k+1} = B_k(x_k, b_k, u_k, x_{k+1}),$$

where B_k is an appropriate function, which can be viewed as a recursive estimator of θ. There are several approaches to implement this estimator (perhaps with some approximation error), including the use of Bayes' rule and the simulation-based method of particle filtering.

The preceding mathematical model forms the basis for a classical adaptive control formulation, where each θ^i represents a set of system parameters, and the computation of the belief probabilities $b_{k,i}$ can be viewed as the outcome of a system identification algorithm. In this context, the problem becomes one of *dual control*, a classical type of combined identification and control problem, whose optimal solution is notoriously difficult.

Another interesting context arises in search problems, where θ specifies the locations of one or more objects of interest within a given space. Some puzzles, including the popular Wordle game, fall within this category, as we will discuss briefly later in this section.

The Exact DP Algorithm - Approximation in Value Space

We will now describe an exact DP algorithm that operates in the space of information vectors I_k. To describe this algorithm, let us denote by $J_k(I_k)$ the optimal cost starting at information vector I_k at time k. We can view I_k as a state of the POMDP, which evolves over time according to the equation

$$I_{k+1} = (I_k, x_{k+1}, u_k) = \big(I_k, f_k(x_k, \theta, u_k, w_k), u_k\big).$$

Viewing this as a system equation, whose right hand side involves the state I_k, the control u_k, and the disturbance w_k, the DP algorithm takes the form

$$\begin{aligned} J_k^*(I_k) &= \min_{u_k \in U_k(x_k)} E_{\theta, w_k}\Big\{g_k(x_k, \theta, u_k, w_k) + J_{k+1}^*(I_{k+1}) \mid I_k, u_k\Big\} \\ &= \min_{u_k \in U_k(x_k)} E_{\theta, w_k}\Big\{g_k(x_k, \theta, u_k, w_k) + \\ &\qquad\qquad J_{k+1}^*\big(I_k, f_k(x_k, \theta, u_k, w_k), u_k\big) \mid I_k, u_k\Big\}, \end{aligned} \tag{2.101}$$

for $k = 0, \ldots, N-1$, with $J_N(I_N) = g_N(x_N)$; see e.g., the DP textbook [Ber17a], Section 4.1.

The algorithm (2.101) is typically very hard to implement, in part because of the dependence of J_{k+1}^* on the entire information vector I_{k+1}, which expands in size according to

$$I_{k+1} = (I_k, x_{k+1}, u_k).$$

To address this difficulty, we may use approximation in value space, based on replacing $J_{k+1}^*(I_{k+1})$ in the DP algorithm (2.101) with some function $\tilde{J}_{k+1}(I_{k+1})$ such that the expected value

$$E_{\theta, w_k}\Big\{\tilde{J}_{k+1}(I_{k+1}) \mid I_k, u_k\Big\} \tag{2.102}$$

can be obtained with a tractable computation for any (I_k, u_k). A useful possibility arises when the cost function approximations

$$E_{w_k}\Big\{\tilde{J}_{k+1}(I_{k+1}) \mid I_k, \theta^i, u_k\Big\}$$

can be obtained for each *fixed value of* θ^i with a tractable computation. In this case, we may compute the cost function approximation (2.102) by using the formula

$$E_{\theta, w_k}\Big\{\tilde{J}_{k+1}(I_{k+1}) \mid I_k, u_k\Big\} = \sum_{i=1}^{m} b_{k,i} E_{w_k}\big\{\tilde{J}_{k+1}(I_{k+1}) \mid I_k, \theta, u_k\big\}$$

which follows from the law of iterated expectations,

$$E_{\theta,w_k}\{\cdot \mid I_k, u_k\} = E_\theta\big\{E_{w_k}\{\cdot \mid I_k, \theta, u_k\} \mid I_k, u_k\big\}.$$

We will now discuss some choices of functions $\tilde{J}_{k+1}$ with a structure that facilitates the implementation of the corresponding approximation in value space scheme. One possibility is to use the optimal cost functions corresponding to the m parameters θ^i,

$$\hat{J}^i_{k+1}(x_{k+1}), \qquad i = 1, \ldots, m. \tag{2.103}$$

In particular, $\hat{J}^i_{k+1}(x_{k+1})$ is the optimal cost that would be obtained starting from state x_{k+1} under the assumption that $\theta = \theta^i$; this corresponds to a perfect state information problem. Then an approximation in value space scheme with one-step lookahead minimization is given by

$$\tilde{u}_k \in \arg \min_{u_k \in U_k(x_k)} \sum_{i=1}^{m} b_{k,i} E_{w_k}\Big\{g_k(x_k, \theta^i, u_k, w_k) + \hat{J}^i_{k+1}\big(f_k(x_k, \theta^i, u_k, w_k)\big) \mid x_k, \theta^i, u_k\Big\}. \tag{2.104}$$

In particular, instead of the optimal control, which minimizes the optimal Q-factor of (I_k, u_k) appearing in the right side of Eq. (2.101), we apply control $\tilde{u}_k$ that minimizes the expected value over θ of the optimal Q-factors that correspond to *fixed values of* θ.

In the case where the horizon is infinite, it is reasonable to expect that an improving estimate of the parameter θ can be obtained over time, and that with a suitable estimation scheme, it converges asymptotically to the correct value of θ, call it θ^*, i.e.,

$$\lim_{k\to\infty} b_{k,i} = \begin{cases} 1 & \text{if } \theta^i = \theta^*, \\ 0 & \text{if } \theta^i \neq \theta^*. \end{cases}$$

Then it can be seen that the generated one-step lookahead controls $\tilde{u}_k$ are asymptotically obtained from the Bellman equation that corresponds to the correct parameter θ^*, and are typically optimal in some asymptotic sense. Schemes of this type have been extensively discussed in the adaptive control literature since the 70s; see the end-of-chapter references and discussion.

Generally, the optimal costs $\hat{J}^i_{k+1}(x_{k+1})$ of Eq. (2.103), which correspond to the different parameter values θ^i, may be hard to compute, despite the fact that they correspond to perfect state information problems.† An alternative possibility is to use off-line trained approximations to $\hat{J}^i_{k+1}(x_{k+1})$ involving neural networks or other approximation architectures. Still another possibility, described next, is to use a rollout approach.

† In favorable special cases, such as linear quadratic problems, the optimal costs $\hat{J}^i_{k+1}(x_{k+1})$ may be easily calculated in closed form. Still, however, even in such cases the calculation of the belief probabilities $b_{k,i}$ may not be simple, and may require the use of a system identification algorithm.

Rollout and Cost Improvement

A simpler possibility for approximation in value space is to use the cost of a given policy π^i in place of the optimal cost $\hat{J}^i_{k+1}(x_{k+1})$ of Eq. (2.103) that corresponds to θ^i. In this case the one-step lookahead scheme (2.104) takes the form

$$\tilde{u}_k \in \arg \min_{u_k \in U_k(x_k)} \sum_{i=1}^{m} b_{k,i} E_{w_k} \Big\{ g_k(x_k, \theta^i, u_k, w_k) + \hat{J}^i_{k+1,\pi^i}\big(f_k(x_k, \theta^i, u_k, w_k)\big) \mid x_k, \theta^i, u_k \Big\}, \tag{2.105}$$

with $\pi^i = \{\mu^i_0, \ldots, \mu^i_{N-1}\}$, $i = 1, \ldots, m$, being known policies, *with components μ^i_k that depend only on x_k*. Here, the term

$$\hat{J}^i_{k+1,\pi^i}\big(f_k(x_k, \theta^i, u_k, w_k)\big)$$

in Eq. (2.105) is the cost of the base policy π^i, calculated starting from the next state

$$x_{k+1} = f_k(x_k, \theta^i, u_k, w_k),$$

under the assumption that θ will stay fixed at the value $\theta = \theta^i$ until the end of the horizon. Note that the cost function of π^i, conditioned on $\theta = \theta^i$, x_k, and u_k, which is needed in Eq. (2.105), can be calculated by Monte Carlo simulation. This is made possible by the fact that the components μ^i_k of π^i depend only on x_k [rather than I_k or the belief state (x_k, b_k)].

The preceding scheme has the character of a rollout algorithm, but strictly speaking, it does not qualify as a rollout algorithm because the policy components μ^i_k involve a dependence on i in addition to the dependence on x_k. On the other hand if we restrict all the policies π^i to be the same for all i, the corresponding functions μ_k depend only on x_k and not on i, thus defining a legitimate base policy. In this case the rollout scheme (2.105) amounts to replacing

$$E_{w_k}\big\{ J^*_{k+1}(I_{k+1}) \mid I_k, u_k \big\}$$

in the DP algorithm (2.101) with

$$E_{w_k}\big\{ J_{k+1,\pi}(I_{k+1}) \mid I_k, u_k \big\}.$$

Similar to Section 2.7, a cost improvement property can then be shown.

Within our rollout context, a policy π such that $\pi^i = \pi$ for all i should be a *robust* policy, in the sense that it should work adequately well for all parameter values θ^i. The method to obtain such a policy is likely problem-dependent. On the other hand robust policies have a long history in the context of adaptive control, and have been discussed widely (see e.g., the book by Jiang and Jiang [JiJ17], and the references quoted therein).

The Case of a Deterministic System

Let us now consider the case where the system (2.100) is deterministic of the form

$$x_{k+1} = f_k(x_k, \theta, u_k). \tag{2.106}$$

Then, while the problem still has a stochastic character due to the uncertainty about the value of θ, the DP algorithm (2.101) and its approximation in value space counterparts are greatly simplified because there is no expectation over w_k to contend with. Indeed, given a state x_k, a parameter θ^i, and a control u_k, the on-line computation of the control of the rollout-like algorithm (2.105), takes the form

$$\tilde{u}_k \in \arg \min_{u_k \in U_k(x_k)} \sum_{i=1}^{m} b_{k,i} \Big(g_k(x_k, \theta^i, u_k) + \hat{J}^i_{k+1,\pi^i}\big(f_k(x_k, \theta^i, u_k)\big) \Big). \tag{2.107}$$

The computation of $\hat{J}^i_{k+1,\pi^i}\big(f_k(x_k, \theta^i, u_k)\big)$ involves a deterministic propagation from the state x_{k+1} of Eq. (2.106) up to the end of the horizon, using the base policy π^i, while assuming that θ is fixed at the value θ^i.

In particular, the term

$$Q_k(x_k, u_k, \theta^i) = g_k(x_k, \theta^i, u_k) + \hat{J}^i_{k+1,\pi^i}\big(f_k(x_k, \theta^i, u_k)\big) \tag{2.108}$$

appearing on the right side of Eq. (2.107) is viewed as a Q-factor that must be computed for every pair (u_k, θ^i), $u_k \in U_k(x_k)$, $i = 1, \ldots, m$, using the base policy π^i. The expected value of this Q-factor,

$$\hat{Q}_k(x_k, u_k) = \sum_{i=1}^{m} b_{k,i} Q_k(x_k, u_k, \theta^i),$$

must then be calculated for every $u_k \in U_k(x_k)$, and the computation of the rollout control $\tilde{u}_k$ is obtained from the minimization

$$\tilde{u}_k \in \arg \min_{u_k \in U_k(x_k)} \hat{Q}_k(x_k, u_k); \tag{2.109}$$

cf. Eq. (2.107). This computation is illustrated in Fig. 2.11.2.

The case of a deterministic system is particularly interesting because we can typically expect that the true parameter θ^* is identified in a finite number of stages, since at each stage k, we are receiving a noiseless measurement relating to θ, namely the state x_k. Once this happens, the problem becomes one of perfect state information.

An illustration similar to the one of Fig. 2.11.2 applies to the rollout scheme (2.105) for the case of a stochastic system. In this case, a Q-factor

$$Q_k(x_k, u_k, \theta^i, w_k) = g_k(x_k, \theta^i, u_k, w_k) + \hat{J}^i_{k+1,\pi^i}\big(f_k(x_k, \theta^i, u_k, w_k)\big)$$

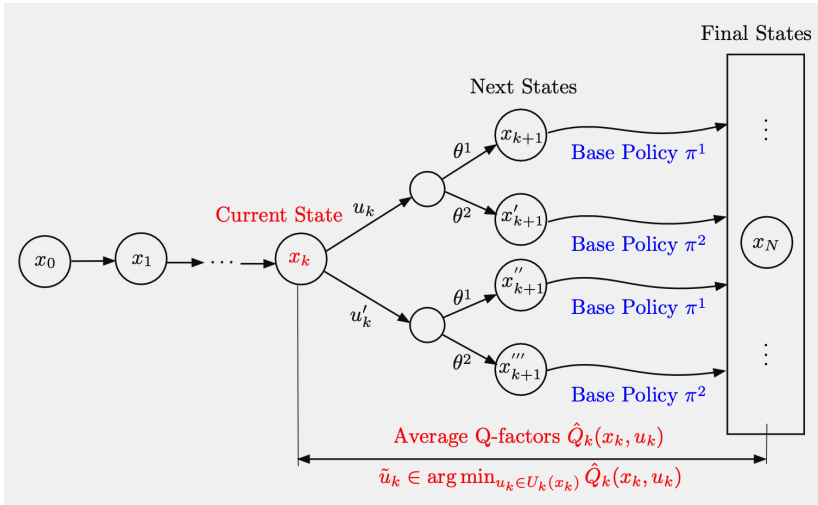

Figure 2.11.2 Schematic illustration of adaptive control by rollout for deterministic systems; cf. Eqs. (2.108) and (2.109). The Q-factors $Q_k(x_k, u_k, \theta^i)$ are averaged over θ^i, using the current belief distribution b_k, and the control applied is the one that minimizes over $u_k \in U_k(x_k)$ the averaged Q-factor

$$\hat{Q}_k(x_k, u_k) = \sum_{i=1}^{m} b_{k,i} Q_k(x_k, u_k, \theta^i).$$

must be calculated for every triplet (u_k, θ^i, w_k), using the base policy π^i. The rollout control $\tilde{u}_k$ is obtained by minimizing the expected value of this Q-factor [averaged using the distribution of (θ, w_k)]; cf. Eq. (2.105).

An interesting and intuitive example that demonstrates the deterministic system case is the popular Wordle puzzle.

Example 2.11.1 (The Wordle Puzzle)

In the classical form of this puzzle, we try to guess a mystery word θ^* out of a known finite collection of 5-letter words. This is done with sequential guesses each of which provides additional information on the correct word θ^*, by using certain given rules to shrink the current mystery list (the smallest list that contains θ^*, based on the currently available information). The objective is to minimize the number of guesses to find θ^* (using more than 6 guesses is considered to be a loss). This type of puzzle descends from the classical family of Mastermind puzzles that centers around decoding a secret sequence of objects (e.g., letters or colors) using partial observations.

The rules for shrinking the mystery list relate to the common letters between the word guesses and the mystery word θ^*, and they will not be described here (there is a large literature regarding the Wordle puzzle). More-

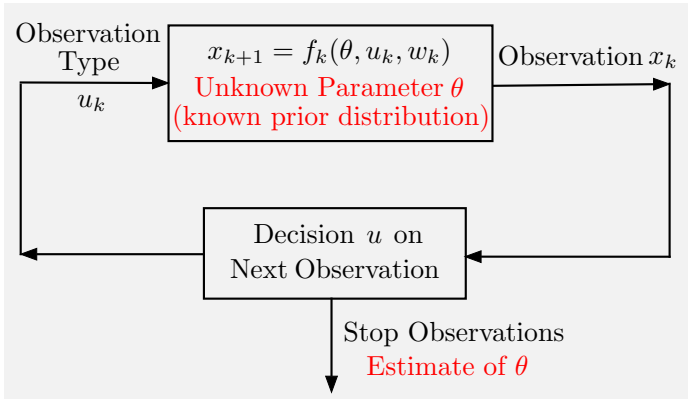

Figure 2.11.3 A view of sequential estimation as an adaptive control problem. The system function f_k does not depend on the current state x_k, so the system provides a decision-dependent noisy observation of θ.

over, θ^* is assumed to be chosen from the initial collection of 5-letter words according to a uniform distribution. Under this assumption, it can be shown that the belief distribution b_k at stage k continues to be uniform over the mystery list. As a result, we may use as state x_k the mystery list at stage k, which evolves deterministically according to an equation of the form (2.106), where u_k is the guess word at stage k. There are several base policies to use in the rollout-like algorithm (2.107), which are described in the papers by Bhambri, Bhattacharjee, and Bertsekas [BBB22], [BBB23], together with computational results, which show that the corresponding rollout algorithm (2.107) performs remarkably close to the optimal policy (first obtained with a very computationally intensive exact DP calculation by Selby in 2022).

The rollout approach also applies to several variations of the Wordle puzzle. Such variations may include for example a larger length $\ell > 5$ of mystery words, and/or a known nonuniform distribution over the initial collection of ℓ-letter words; see [BBB22].

The Case of Sequential Estimation - Alternative Base Policies

We finally note that the adaptive control framework of this section contains as a special case the sequential estimation framework of the preceding section. Here the problem formulation involves a dynamic system of the form

$$x_{k+1} = f_k(\theta, u_k, w_k),$$

where the state x_{k+1} is the observation at time $k+1$ and exhibits no explicit dependence on the preceding observation x_k, but depends on the stochastic disturbance w_k, and on the decision u_k; cf. Figs. 2.11.1 and 2.11.3. This decision may involve a cost and determines the type of next observation out of a collection of possible types.

While the rollout methodology of the present section applies to sequential estimation problems, other rollout algorithms may also be used, depending on the problem's detailed structure. In particular, the rollout algorithms for Bayesian optimization of the works noted in Section 2.10 involve base policies that depend on the current belief state b_k, rather than the current state x_k. Another example of rollout for adaptive control, which uses a base policy that depends on the current belief state is given in Section 6.7 of the book [Ber22a]. For work on related stochastic optimal control problems that involve observation costs and the rollout approach, see Antunes and Heemels [AnH14], and Khashooei, Antunes, and Heemels [KAH15].

2.12 MINIMAX CONTROL AND REINFORCEMENT LEARNING

The problem of optimal control of uncertain systems is usually treated within a stochastic framework, whereby all disturbances w_k are described by probability distributions, and the expected value of the cost is minimized. However, in many practical situations a stochastic description of the disturbances may not be available, but one may have information with less detailed structure, such as bounds on their magnitude. In other words, one may know a set within which the disturbances are known to lie, but may not know the corresponding probability distribution. Under these circumstances we can use a minimax approach, i.e., try to minimize a cost function assuming that the worst possible values of the disturbances will occur. In this approach, we assume an antagonistic opponent, called the *maximizer*, who chooses w_k with the aim to maximize the cost. The controller, who chooses the controls u_k with the aim to minimize the cost, will be referred to as the *minimizer*.†

In this section, we consider a variety of RL schemes for minimax control. We start with exact DP and the corresponding Bellman equation for finite and infinite horizon problems, in order to provide the foundation for approximate DP/RL approaches. The main difference from the stochastic control problems we have discussed earlier is that the disturbance choices w_k are made by maximization rather than by randomization. Accordingly, the expected value operation is replaced by maximization over w_k in the

† The minimax approach to decision and control has its origins in the 50s and 60s. It is also referred to by other names, depending on the underlying context, such as *robust control*, *robust optimization*, *control with a set membership description of the uncertainty*, and *games against nature*. In this book, we will be using the "minimax control" name. The minimax approach is also connected with two-player games, when in lack of information about the opponent, we adopt a worst case viewpoint during on-line play, as well as with contexts where we wish to guard against adversarial attacks.

DP algorithm and in Bellman's equation. We then turn to approximate DP and we discuss two distinct RL schemes.

The first RL scheme (Section 2.12.2) is similar to the ones we have considered so far in this chapter. We approximate the optimal cost function J^* (or cost functions $J_0^*, \ldots, J_{N-1}^*$, in the case of a finite horizon) by an approximation $\tilde{J}$ (or approximations $\tilde{J}_0, \ldots, \tilde{J}_{N-1}$, respectively), and we assume that the minimizer chooses the control u_k by one-step or multistep lookahead minimization, while the maximizer chooses the disturbances by exact cost maximization. Rollout is the special case where $\tilde{J}$ is equal to the cost function of some policy for the minimizer.

The second RL scheme (Section 2.12.3) is based on approximation in policy space, in addition to approximation in value space. We again introduce a cost function approximation $\tilde{J}$. However, instead of choice by maximization, we introduce a policy $\nu(x, u)$ for the maximizer, i.e., a rule by which the disturbance w is chosen at state-control pair (x, u). Naturally, the maximizer's policy ν is chosen to emulate approximately maximizing selections, and we will discuss a few possibilities along this line. However, once the maximizer's policy ν has been chosen, the minimizer's problem becomes a one-player optimization that can be dealt with by using the methods that we have discussed so far in this chapter. This brings to bear the full spectrum of approximation in value space techniques of the preceding sections, including problem approximation and various types of rollout. We discuss briefly an extension of this second RL scheme to distributionally robust control, a set of models that receives much attention at present. Finally, in Section 2.12.4, we illustrate the scheme of Section 2.12.3 within the context of computer chess and other two-person games for which suitable policies ν for the maximizer can be implemented through sophisticated computer engines.

2.12.1 Exact Dynamic Programming for Minimax Problems

Let us first consider a finite horizon case, and assume that the disturbances $w_0, w_1, \ldots, w_{N-1}$ do not have a probabilistic description, but rather are known to belong to corresponding given sets $W_k(x_k, u_k)$, $k = 0, 1, \ldots, N-1$, which may depend on the current state x_k and control u_k. The minimax control problem is to find a policy $\pi = \{\mu_0, \ldots, \mu_{N-1}\}$ with $\mu_k(x_k) \in U_k(x_k)$ for all x_k and k, which minimizes the cost function

$$J_\pi(x_0) = \max_{\substack{w_k \in W_k(x_k, \mu_k(x_k)) \\ k=0,1,\ldots,N-1}} \left[g_N(x_N) + \sum_{k=0}^{N-1} g_k\big(x_k, \mu_k(x_k), w_k\big) \right].$$

The DP algorithm for this problem takes the following form, which resembles the one corresponding to the stochastic DP problem (maximization is used in place of expectation):

$$J_N^*(x_N) = g_N(x_N), \tag{2.110}$$

$$J_k^*(x_k) = \min_{u_k \in U(x_k)} \max_{w_k \in W_k(x_k,u_k)} \Big[g_k(x_k,u_k,w_k) + J_{k+1}^*\big(f_k(x_k,u_k,w_k)\big) \Big]. \tag{2.111}$$

This algorithm can be explained by using a principle of optimality type of argument. In particular, we consider the tail subproblem whereby we are at state x_k at time k, and we wish to minimize the "cost-to-go"

$$\max_{\substack{w_t \in W_t(x_t,\mu_t(x_t)) \\ t=k,k+1,\ldots,N-1}} \left[g_N(x_N) + \sum_{t=k}^{N-1} g_t\big(x_t,\mu_t(x_t),w_t\big) \right].$$

We argue that if $\pi^* = \{\mu_0^*, \mu_1^*, \ldots, \mu_{N-1}^*\}$ is an optimal policy for the minimax problem, then the tail of the policy $\{\mu_k^*, \mu_{k+1}^*, \ldots, \mu_{N-1}^*\}$ is optimal for the tail subproblem. The optimal cost of this subproblem is $J_k^*(x_k)$, as given by the DP algorithm (2.110)-(2.111). The algorithm expresses the intuitive fact that when at state x_k at time k, then regardless of what happened in the past, we should choose u_k that minimizes the worst/maximum value over w_k of the sum of the current stage cost plus the optimal cost of the tail subproblem that starts from the next state. This argument requires a mathematical proof, which turns out to involve a few fine points. For a detailed mathematical derivation, we refer to the author's textbook [Ber17a], Section 1.6. However, the DP algorithm (2.110)-(2.111) is correct assuming finite state and control spaces, among other cases.

Minimax Control and Zero-Sum Game Theory

The theory of minimax control is intimately connected with the theory of dynamic zero-sum game problems, which essentially involve *two* minimax control problems:

(a) The *min-max problem*, where the minimizer chooses a policy first and the maximizer chooses a policy second with knowledge of the minimizer's policy. The DP algorithm for this problem has the form

$$J_N^*(x_N) = g_N(x_N),$$

$$J_k^*(x_k) = \min_{u_k \in U_k(x_k)} \max_{w_k \in W_k(x_k)} \Big[g_k(x_k,u_k,w_k) + J_{k+1}^*\big(f_k(x_k,u_k,w_k)\big) \Big].$$

(b) The *max-min problem*, where the maximizer chooses policy first and the minimizer chooses policy second with knowledge of the maximizer's policy. The DP algorithm for this problem has the form

$$\hat{J}_N(x_N) = g_N(x_N),$$

$$\hat{J}_k(x_k) = \max_{w_k \in W_k(x_k)} \min_{u_k \in U_k(x_k)} \Big[g_k(x_k,u_k,w_k) + \hat{J}_{k+1}\big(f_k(x_k,u_k,w_k)\big) \Big].$$

A basic and easily seen fact is that

$$\text{Max-Min optimal value} \leq \text{Min-Max optimal value}.$$

There is an extensive and time-honored theory for dynamic zero-sum games, which is focused on conditions that guarantee that

$$\text{Max-Min optimal value} = \text{Min-Max optimal value}. \tag{2.112}$$

However, this question is of limited interest in engineering contexts that involve worst-case design. Moreover, the validity of the minimax equality (2.112) is beyond the range of practical RL, and thus will not be discussed here. The main reason is that once approximations are introduced, the delicate assumptions that guarantee the minimax equality are disrupted.

Minimax Control Over an Infinite Horizon

The formulation of the infinite horizon version of the preceding minimax control problem follows similar lines to its stochastic counterpart (cf. Section 1.4). The system equation f, cost per stage g, control constraint sets U, and disturbance constraint sets W do not depend on the time k. For a discounted version of the problem, the cost function of a policy $\pi = \{\mu_0, \mu_1, \ldots\}$ has the form

$$J_\pi(x_0) = \lim_{N\to\infty} \max_{\substack{w_k \in W(x_k,u_k)\\ k=0,1,\ldots}} \left\{ \sum_{k=0}^{N-1} \alpha^k g\big(x_k, \mu_k(x_k), w_k\big) \right\}, \tag{2.113}$$

where $\alpha < 1$ is the discount factor. When the range of values of the stage cost g is bounded, the discount factor guarantees that the limit defining $J_\pi(x_0)$ exists and is finite. It can then be shown that the optimal cost function J^* also takes finite values, and is the unique solution of the Bellman equation

$$J^*(x) = \min_{u\in U(x)} \max_{w\in W(x,u)} \Big[g(x,u,w) + \alpha J^*\big(f(x,u,w)\big)\Big], \qquad \text{for all } x. \tag{2.114}$$

Moreover, a stationary policy μ^* policy is optimal if and only if $\mu^*(x)$ attains the minimum in Bellman's equation for all x. Straightforward analogs of the value and policy iteration algorithms are also valid under the same circumstances.† These results follow from general analyses of abstract DP models under conditions that guarantee that the Bellman operator is a contraction mapping; see the books [Ber12], [Ber22b].

† The policy iteration algorithm involves some subtleties and requires modifications, which have been the subject of quite a bit of research. We will not discuss this issue here; see the author's paper [Ber21c] and book [Ber22b] (Chapter 5).